

## AVL Tree Project Explanation (Java + Swing)

This document explains the core contributions in the AVL Tree Visualizer project — focusing on the implementation of Next/Previous navigation, Balance Factor & Rebalancing logic, Rotation Log system, and Swing Components used in the UI. Each section includes the relevant code snippets, technical explanations, and a simplified way to explain it to the teacher.

### 1. Next / Previous Button Functionality

Code Snippet:

```
// Action listeners for Next and Previous buttons
nextButton.addActionListener(e -> avl.nextStep());
prevButton.addActionListener(e -> avl.prevStep());
```

```
// Inside AVLTree class
private int currentStep = 0;
private List<Node> steps = new ArrayList<>();
```

```
void nextStep() {
    if (currentStep < steps.size() - 1) {
        currentStep++;
        root = steps.get(currentStep);
        notifyChange();
    }
}
```

```
void prevStep() {
    if (currentStep > 0) {
        currentStep--;
        root = steps.get(currentStep);
        notifyChange();
    }
}
```

Explanation:

The Next and Previous buttons are connected to the AVLTree class through action listeners. These buttons let the user move forward or backward between the tree's insertion steps. Each state of the AVL tree is stored in the 'steps' list. The currentStep variable keeps track of

which version of the tree is being shown. When 'Next' or 'Previous' is clicked, the root node changes to the respective step and notifies the visualizer to repaint the tree.

How to explain to teacher:

“Ma’am/Sir, I implemented the Next and Previous button logic. Each time a node is inserted, the state of the tree is saved as a step. Using the two buttons, we can go forward or backward between these tree versions. This helps visualize the AVL balancing process interactively.”

## 2. Balance Factor & Rebalancing Logic

Code Snippet:

```
int height(Node n) { return n == null ? 0 : n.height; }
int balance(Node n) { return n == null ? 0 : height(n.left) - height(n.right); }

void updateHeight(Node n) {
    if (n != null)
        n.height = 1 + Math.max(height(n.left), height(n.right));
}

Node rebalance(Node n) {
    updateHeight(n);
    int bf = balance(n);

    if (bf > 1) {
        if (balance(n.left) < 0) n.left = rotateLeft(n.left); // LR
        n = rotateRight(n); // LL
    } else if (bf < -1) {
        if (balance(n.right) > 0) n.right = rotateRight(n.right); // RL
        n = rotateLeft(n); // RR
    }
    return n;
}
```

Explanation:

The balance factor ensures that the AVL Tree remains balanced after every insertion. It is calculated as (height of left subtree - height of right subtree). If the value is greater than 1 or less than -1, it means the tree is unbalanced. Depending on whether the imbalance is Left-Left, Left-Right, Right-Right, or Right-Left, specific rotations are performed to restore balance. The updateHeight method ensures that every node's height is recalculated after any structural change.

How to explain to teacher:

“The balance factor decides if the tree is balanced or not. If it’s more than +1 or less than -1, the tree needs rebalancing. Based on which side is heavier, rotations are applied. This keeps searching and insertion operations efficient, maintaining  $O(\log n)$  time complexity.”

### 3. Rotation Log System

Code Snippet:

```
private List<String> rotationLog = new ArrayList<>();

private void recordRotation(String type, String reason) {
    rotationLog.add("Rotation: " + type +
        "\nPurpose: Rebalance the tree so heights differ by at most 1." +
        "\nWhy: " + reason + "\n");
}

String getLastRotationExplanation() {
    if (rotationLog.isEmpty()) return "No rotation required for the last operation.";
    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < rotationLog.size(); i++) {
        sb.append("Step ").append(i + 1).append(":\n");
        sb.append(rotationLog.get(i));
        if (i != rotationLog.size() - 1) sb.append("\n");
    }
    return sb.toString();
}
```

Explanation:

The rotationLog system records every rotation that happens during insertion. It stores the type of rotation (LL, LR, RR, RL) and why it was applied. This log is later shown in the explanation area of the GUI, helping users understand what structural change occurred. It improves visualization and learning by providing reasoning for each balancing step.

How to explain to teacher:

“Whenever a rotation happens, the system saves a short explanation — what type of rotation it was and why it was needed. It helps anyone using the visualizer to see not only the tree but also understand the balancing process in text form.”

### 4. Swing Components and Layout

Code Snippet:

```
JFrame frame = new JFrame("AVL Tree Visualizer");
JPanel controlPanel = new JPanel();
JButton nextButton = new JButton("Next");
JButton prevButton = new JButton("Previous");
JTextArea explanationArea = new JTextArea(5, 40);
JScrollPane scrollPane = new JScrollPane(explanationArea);

controlPanel.add(prevButton);
controlPanel.add(nextButton);
frame.add(controlPanel, BorderLayout.SOUTH);
frame.add(scrollPane, BorderLayout.EAST);
frame.setVisible(true);
```

Explanation:

Swing is Java's GUI toolkit. JFrame is the main window container. JPanel holds interactive elements like buttons. JButton is used for user input (Next/Prev). JTextArea displays multiline text for rotation explanations. JScrollPane adds scrolling ability to the text area. The BorderLayout divides the window into regions — buttons are placed at the bottom (SOUTH) and explanations to the right (EAST).

How to explain to teacher:

"I used basic Swing components like JFrame, JPanel, JButton, and JTextArea to create the GUI. The layout is divided using BorderLayout — buttons below and rotation explanations on the right. Swing makes it easy to visualize real-time changes like insertions and rebalancing."

## Conclusion

These sections collectively form the core logic and user interaction part of the AVL Tree Visualizer. The Next/Previous functionality provides navigation between steps, the balance factor and rebalancing logic ensures AVL properties are maintained, the rotation log explains balancing actions, and Swing components create a structured and interactive interface.